

PE-EC802C — VLSI Design Automation

Semester VIII • 3L:0T:0P • 3 Credits

Module 3: Floorplanning & Routing

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Circuit Floorplanning	2
1.1	Slicing vs. Non-Slicing Floorplans	2
1.2	Polish Expressions and Slicing Trees	2
1.3	Shape Functions and Sizing Optimization	2
1.4	Non-Slicing Floorplan Representations	3
2	Circuit Routing	3
2.1	Classification of Routing Problems & Methods	3
2.2	Global Routing & Algorithms	4
2.2.1	Key Global Routing Algorithms	4
2.3	Lee's Maze Routing Algorithm	4
2.3.1	Algorithmic Flow	4
2.3.2	Limitations of Lee's Router	5
2.4	Line-Probe Area Routing	5
3	Channel Routing	5
3.1	Left-Edge Algorithm	6
3.2	Horizontal and Vertical Constraint Graphs	6

1. Circuit Floorplanning

Circuit Floorplanning

Floorplanning is the process of identifying the physical shapes, aspect ratios, and positions of the major functional blocks (macros) on the silicon die. It optimizes the chip area and estimates the total wirelength prior to the cell placement phase.

1.1 Slicing vs. Non-Slicing Floorplans

- **Slicing Floorplan:** A floorplan that can be obtained by recursively partitioning a rectangle into two smaller rectangles using either a horizontal or vertical cut line.
- **Non-Slicing Floorplan:** A floorplan that cannot be partitioned using recursive bisection lines (contains cyclical structures like a wheel configuration).

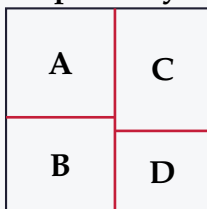
1.2 Polish Expressions and Slicing Trees

Slicing floorplans are represented using two equivalent models:

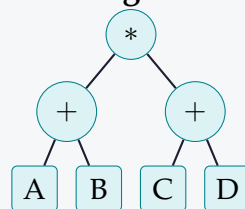
- **Slicing Tree:** A binary tree where the leaf nodes represent the blocks ($A, B, C, \dots$) and the internal nodes represent the cuts ($+$ for horizontal, $*$ for vertical).
- **Polish Expression:** A postfix representation of the slicing tree. A Polish expression is *normalized* if it contains no consecutive identical operators (e.g., $AB++$ is normalized to $AB+C+$).

Slicing Floorplan, Tree, and Polish Expression

Floorplan Layout:



Slicing Tree:



Polish Expression:
 $AB+CD+*$

- **Left Sub:** $AB+$
- **Right Sub:** $CD+$
- **Merged by:** $*$

1.3 Shape Functions and Sizing Optimization

The shape function of a block describes its width w as a function of height h .

- **For a Horizontal Cut ($+$) of blocks 1 and 2:**

$$w = \max(w_1, w_2), \quad h = h_1 + h_2 \quad (1)$$

- **For a Vertical Cut ($*$) of blocks 1 and 2:**

$$w = w_1 + w_2, \quad h = \max(h_1, h_2) \quad (2)$$

Sizing algorithms recursively combine child shape functions at internal tree nodes using horizontal or vertical addition rules, choosing the combination that minimizes total layout area.

1.4 Non-Slicing Floorplan Representations

For non-slicing floorplans (which contain wheel-like cyclic configurations that cannot be cut recursively), binary trees and Polish expressions are insufficient. Common representation models include:

- **Sequence Pair (SP):** Represents a floorplan using a pair of block permutations, e.g., (α, β) . The spatial relation between any two blocks X and Y is defined by their relative order in the two permutations:
 - $(X \text{ is before } Y \text{ in } \alpha) \text{ and } (X \text{ is before } Y \text{ in } \beta) \Rightarrow X \text{ is to the left of } Y.$
 - $(X \text{ is before } Y \text{ in } \alpha) \text{ and } (X \text{ is after } Y \text{ in } \beta) \Rightarrow X \text{ is above } Y.$
- **Bounded Slicing Grid (BSG):** A grid structure of horizontal and vertical line segments that defines placement slots. It allows representing any general topological structure without restricting cuts to slicing boundaries.

2. Circuit Routing

Routing is divided into two sequential steps to manage complexity:

1. **Global Routing:** Divides the chip into a grid of routing channels, planning a coarse pathway for each net through grid regions without assigning specific wire tracks.
2. **Detailed Routing:** Assigns nets to precise metal tracks and layers within the channels planned by the global router.

2.1 Classification of Routing Problems & Methods

Routing problems are classified based on the geometrical constraints and abstraction levels:

- **Local Routing Problems:** Connecting terminal pins within a localized region.
 - **Grid-based Routing:** The routing area is discretized into a uniform grid. Wires can only run along grid tracks, and vias can only be placed at grid intersections. Easy to model but consumes significant memory.
 - **Gridless (Continuous) Routing:** Wires can be placed at any coordinate, restricted only by design rule boundaries (minimum spacing and width). Highly memory efficient and supports variable wire widths.
- **Area Routing:** Connecting pins across a general 2D area (often containing arbitrary blockages and obstacles). Examples include maze routers (Lee's, Soukup's) and line-probe routers (Mikami-Tabuchi, Hightower).
- **Channel Routing:** Connecting pins along two parallel rows (boundaries) using horizontal segments on one layer and vertical segments on another (detailed in Section 4).

2.2 Global Routing & Algorithms

Global routing plans the approximate path of each net to avoid congestion across the chip area. The chip is modeled as a **Grid Graph** $G_c = (V_c, E_c)$, where each vertex v_i represents a routing region (bin) and each edge e_{ij} represents the boundary between adjacent regions with a capacity constraint c_{ij} (maximum number of wires allowed to cross).

2.2.1 Key Global Routing Algorithms

1. **Rectilinear Minimum Steiner Tree (RMST):** A Steiner tree connects a set of pins by introducing auxiliary vertices called **Steiner points** to minimize total wirelength using only horizontal and vertical segments. Finding the RMST is NP-complete, so heuristics (e.g., Hwang's theorem, FLUTE) are used.
2. **Rectilinear Steiner Arborescence (RSA):** A directed Steiner tree routed from a source pin (root) to multiple sink pins such that the path from the root to any sink is a shortest rectilinear path.
3. **Sequential Routing with Rip-up and Re-route:**
 - Router routes nets one by one sequentially (e.g. using Dijkstra's algorithm).
 - Early nets take the shortest paths, causing congestion and capacity violations for later nets.
 - **Rip-up and Re-route:** Congested nets are ripped up, edge weights are increased to penalize congestion, and the nets are rerouted until all capacity constraints are satisfied.
4. **ILP-based Global Routing:** Formulates global routing as a multi-commodity flow problem to route all nets simultaneously, optimizing total wirelength subject to capacity constraints.

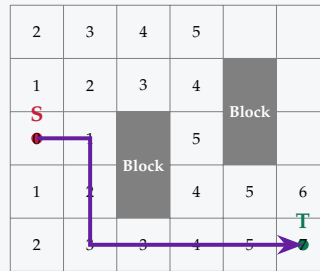
2.3 Lee's Maze Routing Algorithm

Lee's algorithm uses a Breadth-First Search (BFS) grid expansion technique to find the shortest path between a source (S) and target (T) grid cell. It guarantees finding a path if one exists and always finds the shortest path.

2.3.1 Algorithmic Flow

1. **Initialization:** Label the source cell S with 0. Set wave counter $i = 0$.
2. **Wave Expansion:** Find all unlabeled neighbor cells of cells labeled i . If a neighbor is target T , stop. Otherwise, label them $i + 1$. Repeat this expansion until the target is reached or no more cells can be labeled (blocked).
3. **Traceback:** From T , trace back to S by moving to neighbor cells with decreasing labels ($i, i - 1, \dots, 0$).
4. **Clear:** Clear all numbers on the grid.

Lee's Maze Router BFS Grid Expansion



Shortest path found: $S(0) \rightarrow (1) \rightarrow (2) \rightarrow (3) \rightarrow (4) \rightarrow (5) \rightarrow (6) \rightarrow T(7)$. Grid obstacles force the router to loop around the bottom.

2.3.2 Limitations of Lee's Router

- **High Memory Overhead:** Storing wave expansion numbers on a large grid of size $N \times N$ requires $O(N^2)$ memory.
- **Slow Execution:** Expanding wavefronts in all directions behaves like a BFS, taking $O(N^2)$ time.
- **Optimizations:** **Soukup's Router** uses a heuristic search (DFS) towards the target to speed up propagation, falling back to Lee's algorithm only when blocked. **Line-Probe Router** routes lines instead of cells, reducing memory usage.

2.4 Line-Probe Area Routing

To overcome the high memory $O(N^2)$ and execution time constraints of grid-based BFS expansion in Lee's maze router, line-probe algorithms are used:

- **Mikami-Tabuchi Algorithm:** Shoots orthogonal horizontal and vertical probe lines from both the source S and the target T . When a line generated from S intersects any line generated from T , a path is established. It runs in time proportional to the number of probe lines ($O(L)$), drastically reducing search time and memory usage.
- **Hightower's Algorithm:** Similar to Mikami-Tabuchi, but instead of shooting probes from all intersection points, it only shoots escape lines from the boundaries of obstacles obstructing the straight path. It is faster than Mikami-Tabuchi but does not guarantee finding a path even if one exists.

3. Channel Routing

Channel Routing

Channel Routing is a detailed routing technique that connects terminal pins along two parallel rows (top and bottom boundaries of a horizontal channel) using horizontal wire segments on one metal layer and vertical wire segments on a different metal layer.

3.1 Left-Edge Algorithm

The Left-Edge algorithm is a greedy track allocation algorithm that routes nets in a single pass from left to right.

1. Sort all nets in ascending order of their left-most pin x -coordinate.
2. Select the first unrouted net and assign it to the first available track.
3. Scan left to right to find the next unrouted net whose left boundary does not overlap with the previously routed net on the same track. Assign it to the same track.
4. Repeat step 3 until no more nets can fit in the current track.
5. Open a new track and repeat steps 2-4 until all nets are routed.

3.2 Horizontal and Vertical Constraint Graphs

To prevent wire overlaps, the channel router must respect two types of constraints:

- **Horizontal Constraints:** Nets overlapping in their horizontal span cannot share the same track. (Modeled using interval graphs).
- **Vertical Constraints:** If net A 's top pin is directly above net B 's bottom pin at some column, net A 's horizontal wire segment must be placed in a track above net B 's track to avoid vertical wire overlaps. (Represented as a directed **Vertical Constraint Graph (VCG)** where edge $A \rightarrow B$ means $\text{track}(A) > \text{track}(B)$).

Quick Revision — Module 3

Floorplanning & Routing | PE-EC802C

Key Formulas and Metrics at a Glance

Horizontal Cut (+)	$h_{\text{parent}} = h_1 + h_2, \quad w_{\text{parent}} = \max(w_1, w_2)$
Vertical Cut (*)	$w_{\text{parent}} = w_1 + w_2, \quad h_{\text{parent}} = \max(h_1, h_2)$
Lee's Wave Expansion	Label value $L \leq x_s - x_t + y_s - y_t $ (Manhattan limit)
Channel Routing Tracks	Lower Bound on Tracks = $\max_x d(x)$ (d_{max} density)

The maximum channel density d_{max} represents the minimum number of tracks required to route the channel without overlap.

One-Line Definitions

- **Floorplanning:** Finding optimal block locations and aspect ratios prior to cell placement.
- **Slicing Cut:** A cut line that splits a floorplan region completely into two sub-regions.
- **Polish Expression:** A postfix notation string describing slicing sequences and leaf blocks.
- **Shape Function:** A curve or step function representing possible width and height values of a block.
- **Maze Routing:** Routing algorithms that find grid-based paths between pins around layout obstacles.
- **Traceback:** The path reconstruction phase of Lee's algorithm working backward from target to source.
- **Left-Edge Algorithm:** A track allocation algorithm that routes nets sorted by their left-most coordinate.
- **Vertical Constraint Graph:** A directed graph indicating which net must be placed above another to avoid overlaps.

Frequently Asked / Viva Questions

Q1. What is a normalized Polish expression, and why is it important?

A Polish expression is normalized if it has no consecutive identical operators (e.g. $AB++$ is not normalized, whereas $AB+C+$ is). Normalization guarantees a 1-to-1 mapping between Polish expressions and slicing floorplans, which reduces the search space during optimization.

Frequently Asked / Viva Questions (contd.)**Q2. Why is floorplanning done before placement?**

Floorplanning handles large blocks (macros) and defines the global silicon structure. Placing small standard cells directly without placing macros first would lead to congestion, wiring bottlenecks, and poor cell distribution.

Q3. Explain the wave expansion step in Lee's maze routing algorithm.

Starting at the source node, neighbors are labeled with 1. Then, all unlabeled neighbors of 1 are labeled with 2. This process is repeated, labeling neighbors of i with $i + 1$, until the target cell is reached. This BFS propagation guarantees finding the shortest path.

Q4. What are the main drawbacks of Lee's maze router?

(1) It uses $O(N^2)$ memory to store the grid cell states and label numbers. (2) It is slow ($O(N^2)$ time) because the wavefront expands in all directions, looking like a growing circle, which explores many unnecessary grid nodes.

Q5. How does Soukup's router improve on Lee's algorithm?

Soukup's router uses a depth-first search (DFS) strategy to route directly toward the target cell. If it hits an obstacle, it switches to Lee's BFS expansion to find a way around the obstacle, then switches back to DFS. This reduces the search time significantly.

Q6. Explain the vertical constraint in channel routing with an example.

If Net 1 has a pin on the top row of a channel at column 3, and Net 2 has a pin on the bottom row at column 3, the vertical wire segments of Net 1 and Net 2 both lie along column 3. To prevent a short circuit, Net 1's horizontal wire segment must be placed in a track above Net 2's track.

Q7. What is a dogleg in channel routing, and why is it used?

A dogleg is a vertical wire segment that splits a net's horizontal wire segment across different tracks. It is used to resolve cyclical conflicts in Vertical Constraint Graphs, reducing the total number of tracks needed to route the channel.

Q8. How does the Left-Edge algorithm route nets?

It first sorts all nets by their left-most pin coordinates. It then processes nets in order, placing as many non-overlapping nets as possible into the first track. When no more nets can fit, it moves to the next track and repeats the process until all nets are routed.

Q9. What is the difference between slicing and non-slicing floorplans?

A slicing floorplan can be completely partitioned into individual blocks using recursive vertical and horizontal cuts. A non-slicing floorplan cannot be divided this way because it contains wheel-like cycles of blocks.

Q10. What is channel density, and why is it important?

Frequently Asked / Viva Questions (contd.)

Channel density is the maximum number of nets crossing any vertical slice of a routing channel. It represents the mathematical lower bound on the number of tracks required to route the channel without overlaps.

Comparison Snapshot**Slicing vs. Non-Slicing Floorplans:**

Feature	Slicing Floorplan	Non-Slicing Floorplan
Representation	Binary trees, Polish expressions.	Constraint graphs (BSG, Sequence Pairs).
Complexity	Optimization is fast ($O(n \log n)$).	Optimization is slow and computationally complex.
Optimal Sizing	Can be solved optimally in polynomial time.	Sizing is NP-complete.

Global Routing vs. Detailed Routing:

Feature	Global Routing	Detailed Routing
Abstraction	Coarse grid, ignores specific tracks.	Actual grid, routes on precise metal tracks.
Objective	Allocates nets to routing regions to avoid congestion.	Resolves layout design rules, placing vias and wires.
Algorithm	Steiner trees, maze routers, network flows.	Left-Edge algorithm, dogleg routers, grid maze.